

COMP-8171-0 - Database Systems Security

Assignment I Authentication and Access Control

Claire Ko
A01065828

[2 marks] Survey and briefly highlight commonly used practices to prevent piracy and reverse engineering.

Piracy means unauthorized copying, distribution, modification, or use of copyrighted software and digital intellectual property. Reverse engineering refers to the process of taking apart software, hardware, or compiled computer code to see how it works when you do not have the original blueprints or source code. Combine these two scenarios, the main practice to prevent the issue (anti-piracy and anti-reverse engineering) generally aim to enforce authorized use of the software, prevent unauthorized access or modification, and make the application's code and intellectual property more difficult to extract or reuse.

After the survey, there are a few methods that are commonly used to prevent piracy and reverse engineering.

Common anti-piracy approaches:

- License enforcement : time-limited trials, annual licenses, license keys.
- Cryptographic license signing : prevents users from modifying expiry dates or features.
- Machine/device binding : makes a license harder to copy to another installation.
- Online license validation : allows activation, periodic verification, and revocation.

Common anti-reverse-engineering approaches

- Code obfuscation : makes compiled code harder to understand.
- Anti-tamper/integrity checks : makes modifying or removing license checks more difficult.
- Server-side protection : keeps sensitive business logic/IP away from the client

My application uses a layered approach to software protection. License enforcement implements a two-month free trial followed by an annual license. License information is cryptographically protected so that users cannot simply modify the expiration date or enabled features. Obfuscation using Obfuscator makes the compiled .NET application more difficult to reverse engineer and locate or modify the licensing logic. Server-side authentication, authorization, and database operations further reduce the amount of sensitive logic exposed to the client

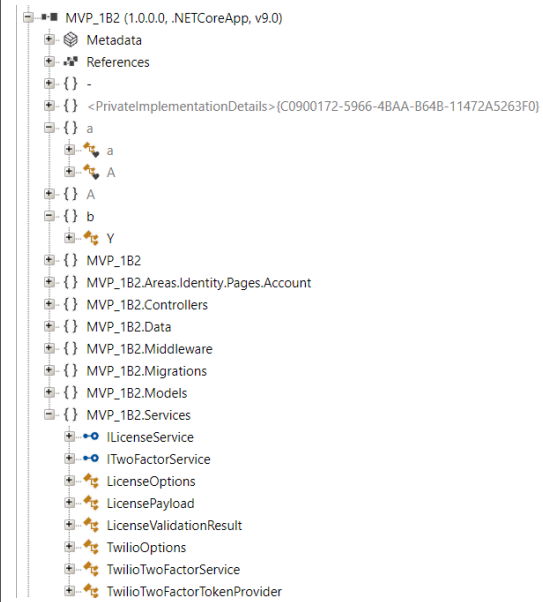
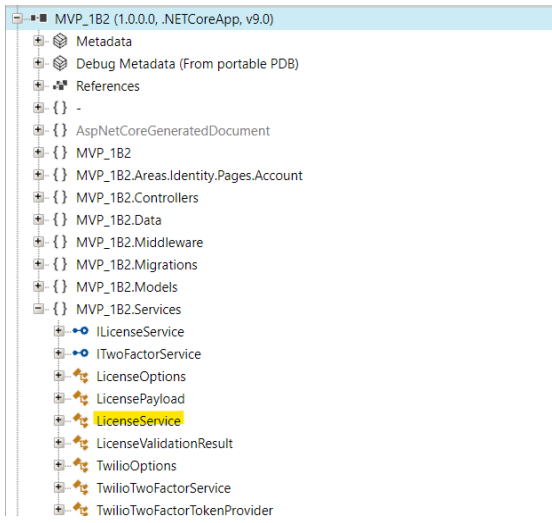
- <https://www.revenera.com/software-monetization/glossary/software-piracy>
- <https://fluidattacks.com/cybersecurity-essentials/what-is-reverse-engineering>

[1 mark] For preventing reverse engineering try out any free obfuscation Nuget package to obfuscate part of your MVC app (perhaps the above license verification functionality)

Obfuscation in ASP.Net Core MVC application could mean making the C# code becomes a .NET assembly such as .dll file. However, .NET assemblies contain metadata that makes them relatively easy to inspect with tools such as ILSpy or dnSpy.

For my application, I use a free .NET obfuscation tool that was used to obfuscate the application's license verification functionality. The LicenseService was selected because it contains security-sensitive licensing logic that could be targeted for modification or analysis.

The application assembly was first inspected using a .NET decompiler before obfuscation. The class and method names were relatively easy to identify and understand. The assembly was then processed using the selected obfuscation tool. After obfuscation, the application continued to perform its license verification functions, while the resulting assembly was significantly less readable and the original code structure was more difficult to identify.

After obfuscation	Before obfuscation
	

LicenseService.cs Before Obfuscation

```
// MVP_1B2, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
// MVP_1B2.Services.LicenseService
using ...

[Obfuscation(Exclude = false, Feature = "all", ApplyToMembers = true, StripAfterObfuscation = true)]
public sealed class LicenseService : ILicenseService
{
    private readonly IHostEnvironment _environment;

    private readonly LicenseOptions _options;

    private readonly ILogger<LicenseService> _logger;

    private static readonly JsonSerializerOptions JsonSerializerOptions = new JsonSerializerOptions
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        WriteIndented = false
    };

    public LicenseService(IHostEnvironment environment, IOption<LicenseOptions> options, ILogger<LicenseService> logger)
    {
        ...
    }

    public LicenseValidationResult Validate()
    {
        ...
    }

    private string ResolvePath(string path)
    {
        ...
    }
}
```

LicenseService.cs After Obfuscation

 Warning: Some assembly references could not be resolved automatically. This might lead to incorrect decompilation of some parts for ex. property getter/setter access. To get optimal decompilation results, please manually add the missing references to the project.

```
// MVP_1B2, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
// b.Y
using ...

public sealed class Y : ILicenseService
{
    private readonly IHostEnvironment m_A;

    private readonly LicenseOptions m_A;

    private readonly ILogger<Y> m_A;

    private static readonly JsonSerializerOptions m_A;

    public Y(IHostEnvironment P_0, IOption<LicenseOptions> P_1, ILogger<Y> P_2)
    {
        ...
    }

    public LicenseValidationResult Validate()
    {
        ...
    }

    private string A(string P_0)
    {
        ...
    }

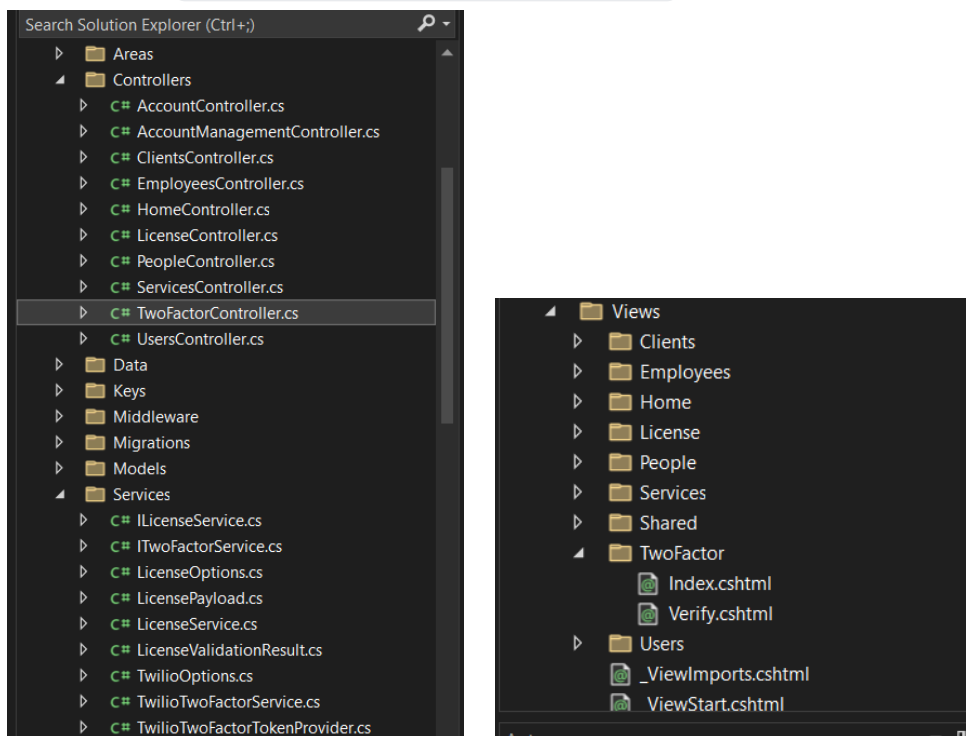
    static Y()
    {
        ...
    }
}
```

[1 marks] Supplement user authentication with 2 factor authentication. Consider using email or Twilio SMS API (or other such service) if they allow some free messages for testing. Include a screen shot of the text message to confirm your implementation.

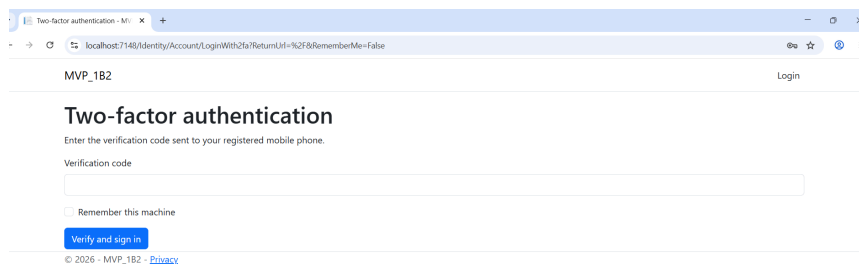
I implement the Twilio SMS to enforce the 2 factor authentication. After login, the 2FA request page shows up for the user to input the verification code which is sent to the registered cellphone number.

To implement 2FA, the TwoFactorController, corresponding Twilio services components, and views are created.

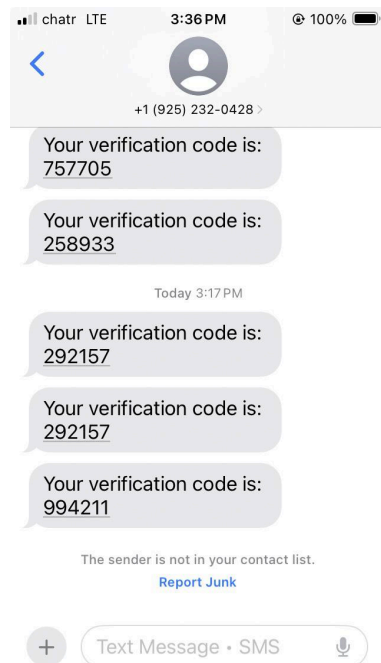
Demo link:  **Recording 2026-09-22 152825.mp4**



2FA UI:



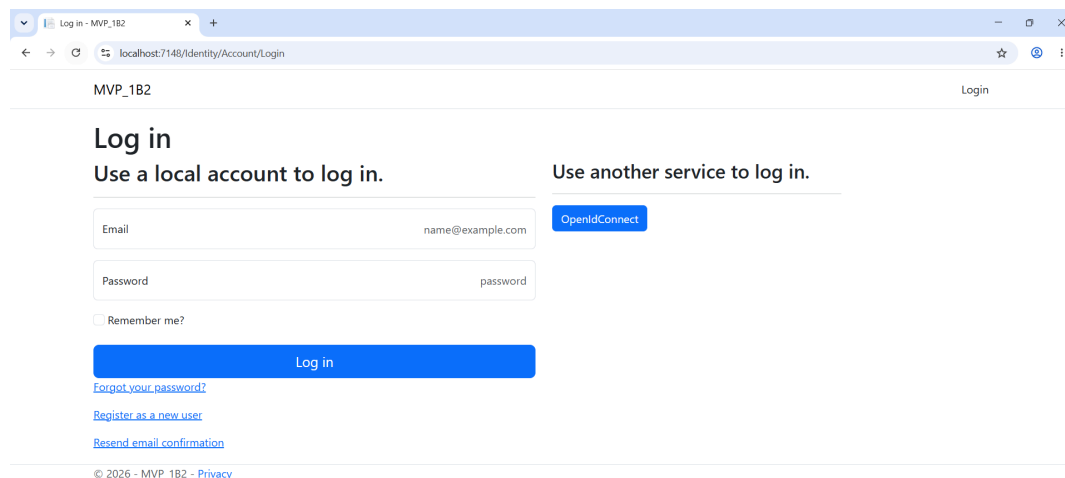
Text Message:



[2 marks] Single Sign ON, preferably with Active Directory or Office 365.

To implement the SSO, an application register is needed to be set up in Azure Portal in order to obtain both Client ID and also Tenant ID. Also I created 2 users from the same domain in order to solve the restriction of the tenant domain problem since my app is configured as **single-tenant**. If I open the application to **multiple tenants**, and use my BCIT account, the administrator consent is required due to different tenants.

Demo Link:  Recording 2026-09-22 154745.mp4



Default Directory | Overview

Microsoft Entra has a simpler, integrated experience for managing all your Identity and Access Management needs. Try the new Microsoft Entra admin center!

Overview Monitoring Properties Recommendations Setup guides

Search your tenant

Basic information

Name	Default Directory	Users	3
Tenant ID	[REDACTED]	Groups	0
Primary domain	colleen1005yahoo.com.onmicrosoft.com	Applications	1
License	Microsoft Entra ID Free	Devices	0

Alerts

Migrate to the converged Authentication methods policy
Please migrate your authentication methods off the legacy MFA and SSPR policies by September 2025 to avoid any service impact.

Default Directory | App registrations

New registration Endpoints Troubleshoot Refresh Download Got feedback?

Azure Active Directory Authentication Library (ADAL) and Azure AD Graph no longer receive new feature updates. We continue to provide technical support and security updates. Applications should be upgraded to Microsoft Authentication Library (MSAL) and Microsoft Graph. [Learn more](#)

All applications Owned applications Deleted applications Deactivated applications Applications from personal account

Start typing a display name or application (client) ID to filter these r... Add filters

1 applications found

Display name	Application (client) ID	Created on	Certificates & secrets
MVP_1B2	[REDACTED]	2026-09-18	-

Users

New user Edit Delete Download users (Preview) Bulk operations Refresh Manage view Per-user MFA Got feedback?

Search users Add filter

3 users found

Display name	User principal name	User type	Is Agent	On-premises sy...	Sign-in identifiers	Company name
Claire Ko	colleen1005.yaho...	Member	No	No	View	
MVP Test User	testuser@colleen...	Member	No	No	View	
Test User 2	testuser2@collee...	Member	No	No	View	

[1 mark] Use SSL with client side certificate verification (in addition to the default server-side certificate verification).

The implementation of the SSL with client side certificate requires following steps:

1. **Configure HTTPS in ASP.NET Core** - The MVC application uses HTTPS endpoints. Port 7148 provides normal HTTPS and the port 7443 is configured specifically for mTLS.
2. **Configure the server to require a client certificate**

```
// HTTPS + Client Certificate endpoint
// Security demonstration
// -----
options.ListenAnyIP(
    7443,
    listenOptions =>
    {
        listenOptions.UseHttps(
            httpsOptions =>
            {
                httpsOptions.ClientCertificateMode =
                    ClientCertificateMode.RequireCertificate;

                // TLS 1.2 for easier Wireshark demonstration
                httpsOptions.SslProtocols =
                    SslProtocols.Tls12;

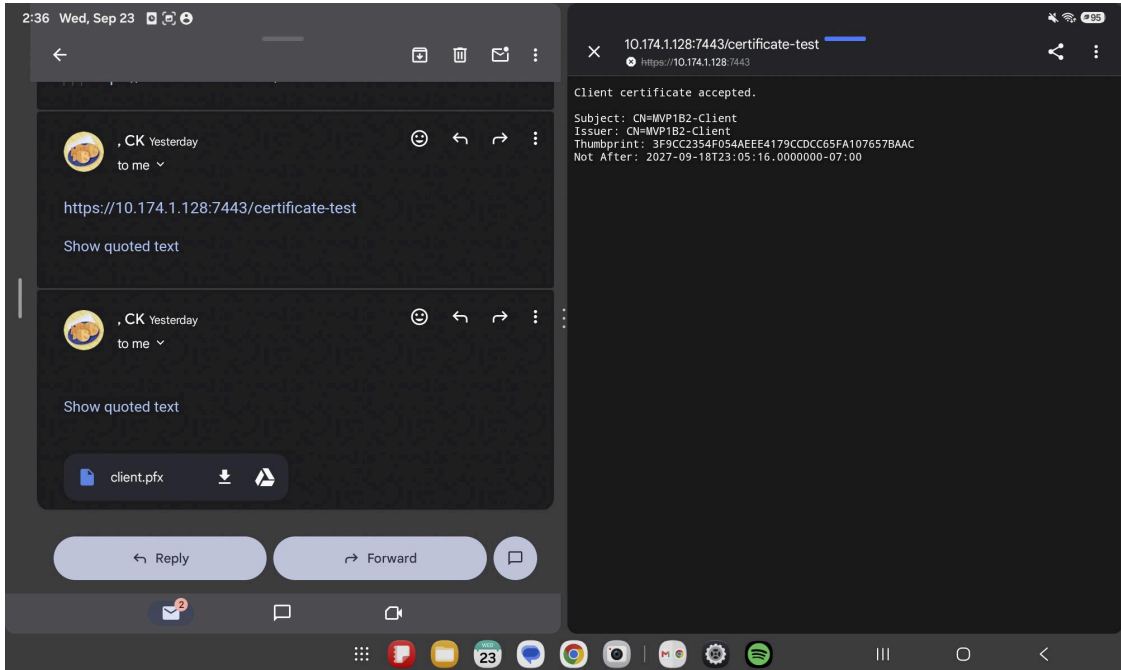
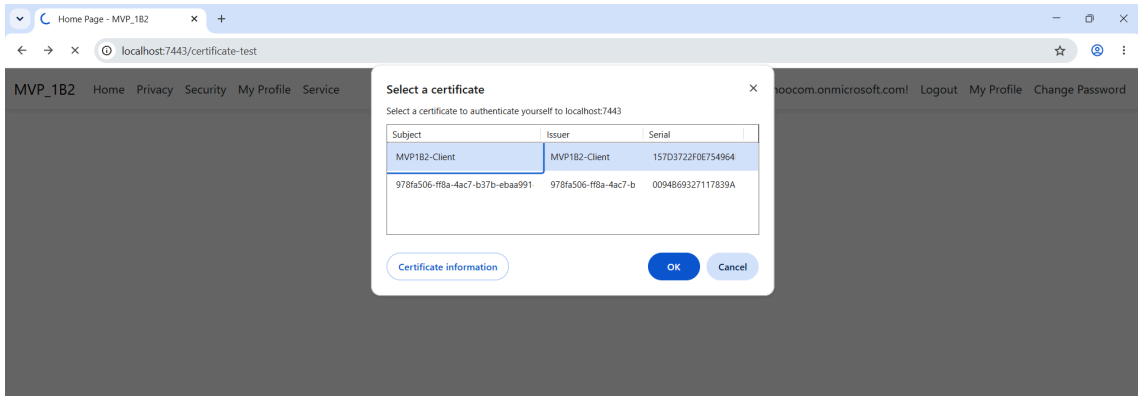
                var expectedThumbprint =
                    builder.Configuration[
                        "ClientCertificate:Thumbprint"];
            }
        );
    }
);
```

3. **Create and install the client certificate** - I installed the private key that was generated and exported as a .pfx file in my tablet as a client device.
4. **Connect the tablet to the server** - Tablet link to : <https://localhost:7443/certificate-test>, it will ask to select the proper certificate first, select the proper one (MVP1B2).

Then I turned on the wireshark to capture wifi, then filter the signal with **tcp.port == 7443**. We could observe the client - server connection with a handshake process. The certificate name and information is also clearly identified in the wireshark package information.

No.	Source	Destination	Protocol	Length	Header checksum status	Info
80	10.174.1.127	10.174.1.128	TCP	74	Unverified	34888 → 7443 [SYN] Seq=1604848527 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=3951386973 TSecr=0 WS=512
81	10.174.1.128	10.174.1.127	TCP	74	Unverified	7443 → 34888 [SYN, ACK] Seq=3259143959 Ack=1604848528 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=340732980 TSecr=340732980
82	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1604848528 Ack=3259143960 Win=65536 Len=0 TSval=3951386993 TSecr=340732980
83	10.174.1.127	10.174.1.128	TCP	1514	Unverified	34888 → 7443 [ACK] Seq=1604848528 Ack=3259143960 Win=65536 Len=1448 TSval=3951387000 TSecr=340732980 [TCP PDU reassembled]
84	10.174.1.127	10.174.1.128	TLSv1.2	415	Unverified	Client Hello
85	10.174.1.128	10.174.1.127	TCP	66	Unverified	7443 → 34888 [ACK] Seq=3259143968 Ack=1604850325 Win=65280 Len=0 TSval=340733011 TSecr=3951387000
86	10.174.1.127	10.174.1.128	TCP	415	Unverified	[TCP Segment Not Reassembled] 34888 → 7443 [PSH, ACK] Seq=1604848528 Ack=3259143968 Win=65536 Len=349 TSval=3951387000 TSecr=340733011
87	10.174.1.127	10.174.1.128	TCP	78	Unverified	[TCP Dup ACK 48941] 7443 → 34888 [ACK] Seq=3259143968 Ack=1604850325 Win=65280 Len=0 TSval=340733011 TSecr=3951387000
88	10.174.1.128	10.174.1.127	TLSv1.2	1364	Unverified	Server Hello, Certificate[Malformed Packet]
89	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1604850325 Ack=3259145258 Win=68088 Len=0 TSval=3951387095 TSecr=340733081
90	10.174.1.127	10.174.1.128	TLSv1.2	73	Unverified	Alert (Level: Fatal, Description: Certificate Unknown)
91	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1604850325 Ack=3259145258 Win=68088 Len=0 TSval=3951387097 TSecr=340733081
92	10.174.1.128	10.174.1.127	TCP	66	Unverified	7443 → 34888 [ACK] Seq=3259145258 Ack=1604850333 Win=65280 Len=0 TSval=340733088 TSecr=3951387096
93	10.174.1.127	10.174.1.128	TCP	66	Unverified	7443 → 34888 [FIN, ACK] Seq=3259145258 Ack=1604850333 Win=65280 Len=0 TSval=340733093 TSecr=3951387096
94	10.174.1.127	10.174.1.128	TCP	74	Unverified	34888 → 7443 [SYN] Seq=1122444837 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=3951387104 TSecr=0 WS=512
95	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1604850333 Ack=3259145259 Win=68088 Len=0 TSval=3951387106 TSecr=340733093
96	10.174.1.128	10.174.1.127	TCP	74	Unverified	7443 → 34888 [SYN, ACK] Seq=3259145259 Ack=1122444838 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=340733097 TSecr=3951387106
97	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1122444838 Ack=1261094104 Win=65536 Len=0 TSval=3951387118 TSecr=340733097
98	10.174.1.127	10.174.1.128	TCP	1514	Unverified	34888 → 7443 [ACK] Seq=1122444838 Ack=1261094104 Win=65536 Len=1448 TSval=3951387120 TSecr=340733097 [TCP PDU reassembled]
99	10.174.1.127	10.174.1.128	TLSv1.2	415	Unverified	Client Hello
100	10.174.1.128	10.174.1.127	TCP	66	Unverified	7443 → 34888 [ACK] Seq=1261094194 Ack=1122444635 Win=65280 Len=0 TSval=340733118 TSecr=3951387120
101	10.174.1.127	10.174.1.128	TLSv1.2	1364	Unverified	Server Hello, Certificate[Malformed Packet]
102	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1122444635 Ack=1261095492 Win=68088 Len=0 TSval=3951387134 TSecr=340733120
103	10.174.1.127	10.174.1.128	TLSv1.2	1280	Unverified	Certificate, Client Key Exchange, Certificate Verify, Change Cipher Spec, Encrypted Handshake Message
104	10.174.1.127	10.174.1.128	TLSv1.2	165	Unverified	Application Data
105	10.174.1.127	10.174.1.128	TLSv1.2	580	Unverified	Application Data
106	10.174.1.128	10.174.1.127	TCP	66	Unverified	7443 → 34888 [ACK] Seq=1261095492 Ack=1122444862 Win=65280 Len=0 TSval=340733201 TSecr=3951387197
107	10.174.1.127	10.174.1.128	TLSv1.2	117	Unverified	Change Cipher Spec, Encrypted Handshake Message
108	10.174.1.127	10.174.1.128	TLSv1.2	128	Unverified	Application Data
109	10.174.1.127	10.174.1.128	TLSv1.2	146	Unverified	Application Data, Application Data
110	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1122444862 Ack=1261095543 Win=68088 Len=0 TSval=3951387223 TSecr=340733203
111	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1122444862 Ack=1261095605 Win=68088 Len=0 TSval=3951387223 TSecr=340733209
112	10.174.1.127	10.174.1.128	TCP	66	Unverified	34888 → 7443 [ACK] Seq=1122444862 Ack=1261095685 Win=68088 Len=0 TSval=3951387224 TSecr=340733209
113	10.174.1.127	10.174.1.128	TLSv1.2	104	Unverified	Application Data
114	10.174.1.127	10.174.1.128	TLSv1.2	145	Unverified	Application Data

```
Length: 778
Certificates Length: 775
Certificates (775 bytes)
Certificate Length: 772
Certificate [..]: 30820300308201e0a0030201020210157d3722f0e754964e2c084dcf5d3580300d06092a864886f70d010106050030183116301406035504030b
signedCertificate
version: v3 (2)
serialNumber: 0x157d3722f0e754964e2c084dcf5d3580
signature (sha256WithRSAEncryption)
Algorithm Id: 1.2.840.113549.1.1.11 (sha256WithRSAEncryption)
issuer: rdnSequence (0)
> rdnSequence: 1 item (id-at-commonName=MVP1B2-Client)
validity
> notBefore: utcTime (0)
> notAfter: utcTime (0)
subject: rdnSequence (0)
> rdnSequence: 1 item (id-at-commonName=MVP1B2-Client)
subjectPublicKeyInfo
algorithm (rsaEncryption)
Algorithm Id: 1.2.840.113549.1.1.1 (rsaEncryption)
padding: 0
> subjectPublicKeyInfo [..]: 3082010a0282010100c716b0022246166df00984bdd2222aa79e962cb077e1703175333df2f3db353b49b9dbdf7d4bdd751a
extensions: 3 items
> Extension (id-ce-keyUsage)
> Extension (id-ce-extKeyUsage)
> Extension (id-ce-subjectKeyIdentifier)
algorithmIdentifier (sha256WithRSAEncryption)
Algorithm Id: 1.2.840.113549.1.1.11 (sha256WithRSAEncryption)
padding: 0
encrypted [..]: 81981725639ab025dfed71d9996239d98934ff1b6c3674aaf42e7faed8dc2c957f18bf5f8f066ab24b9ff1eed4cf071abdb8e958f81802d21f
TLV1.2 Record Layer: Handshake Protocol: Client Key Exchange
Certificate Type: Handshake (22)
```



[1 mark] If the client is behind NAT or CGNAT, describe how reverse tunneling helps secure the access? Employ reverse.

Reverse tunneling allows a client or server behind NAT/CGNAT to establish an outbound connection to a publicly accessible tunnel service. Instead of requiring the private device to accept incoming connections or exposing its private IP address, the tunnel forwards authorized external requests through the established connection to the local ASP.NET Core application. This reduces direct exposure of the local network and avoids the need for inbound port forwarding.

In my implementation, I install Cloudflare to provide secure access to the ASP.NET Core application with command: **powershell winget install --id cloudflare.cloudflare**. Then, I hosted the ASP.NET Core application and created a temporary Cloudflare Tunnel by command : **cloudflared tunnel --url https://localhost:7148**. I now could access the application using the public HTTPS URL generated by Cloudflare. The application was successfully accessible from an external device without configuring router port forwarding or exposing the server's private LAN IP address. We could also access the server from other devices as well.

```
No newer package versions are available from the configured sources.
PS C:\Users\coll> cloudflared --version
cloudflared version 2026.9.1 (built 2026-09-10T13:52 UTC)
PS C:\Users\coll> cloudflared tunnel --url https://localhost:7148
2026-09-22T04:37:47Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way to experiment and try it out. However, we are aware that these account-less Tunnels have no uptime guarantee, are subject to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/web-site/terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms. If you intend to use Tunnels in production you should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect-apps
2026-09-22T04:37:47Z INF Requesting new quick Tunnel on trycloudflare.com...
2026-09-22T04:37:50Z INF Your quick Tunnel has been created! Visit it at (it may take some time to be reachable):
2026-09-22T04:37:50Z INF https://und-era-such-mardi.trycloudflare.com
2026-09-22T04:37:50Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/./cloudflared ~/./cloudflare-warp ~/cloudflare-warpp]
2026-09-22T04:37:50Z INF Version 2026.9.1 (Checksum 2837g88cc0f5d58f15b6dc478376de90b4d3ba5241c7947455d1e0aadf429712)
2026-09-22T04:37:50Z INF GOOS: windows, GOModule: gol.26.8, GoArch: amd64
2026-09-22T04:37:50Z INF Settings: map[ha-connections:1 protocol:quic url:https://localhost:7148]
2026-09-22T04:37:50Z INF cloudflared will not automatically update on Windows systems.
2026-09-22T04:37:50Z INF Generated Connector ID: f5250b35-0d7e-4bba-b914-c56f9daeaa26
2026-09-22T04:37:50Z INF Initial protocol quic
2026-09-22T04:37:50Z INF ICMP proxy will use 10.174.1.128 as source for IPv4
2026-09-22T04:37:50Z INF ICMP proxy will use fe80::edd9:947b:3f9e:d4e3 in zone Wi-Fi as source for IPv6
2026-09-22T04:37:50Z INF cloudflared does not support loading the system root certificate pool on Windows. Please use --origin-ca-pool <PATH> to specify the path to the certificate pool
2026-09-22T04:37:50Z INF ICMP proxy will use 10.174.1.128 as source for IPv4
2026-09-22T04:37:50Z INF Tunnel connection curve preferences: [X25519MLKEM768 CurveID(65074) CurveP256] connIndex=0 event=0 ip=198.41.200.33
2026-09-22T04:37:50Z INF
2026-09-22T04:37:50Z INF CONNECTIVITY PRE-CHECKS
2026-09-22T04:37:50Z INF
2026-09-22T04:37:50Z INF COMPONENT TARGET STATUS DETAILS
2026-09-22T04:37:50Z INF DNS Resolution region1.v2.argotunnel.com PASS DNS Resolved successfully
2026-09-22T04:37:50Z INF DNS Resolution region2.v2.argotunnel.com PASS DNS Resolved successfully
2026-09-22T04:37:50Z INF UDP Connectivity region1.v2.argotunnel.com PASS QUIC connection successful
2026-09-22T04:37:50Z INF UDP Connectivity region2.v2.argotunnel.com PASS QUIC connection successful
2026-09-22T04:37:50Z INF TCP Connectivity region1.v2.argotunnel.com PASS HTTP/2 connection successful
2026-09-22T04:37:50Z INF TCP Connectivity region2.v2.argotunnel.com PASS HTTP/2 connection successful
2026-09-22T04:37:50Z INF Cloudflare API api.cloudflare.com/443 PASS API is reachable
2026-09-22T04:37:50Z INF SUMMARY: Environment is healthy, cloudflared will use 'quic' as primary protocol.
2026-09-22T04:37:50Z INF
```

Welcome

Learn about [building Web apps with ASP.NET Core](#).



Welcome

Learn about [building Web apps with ASP.NET Core](#).



Ask Gemini



New Tab

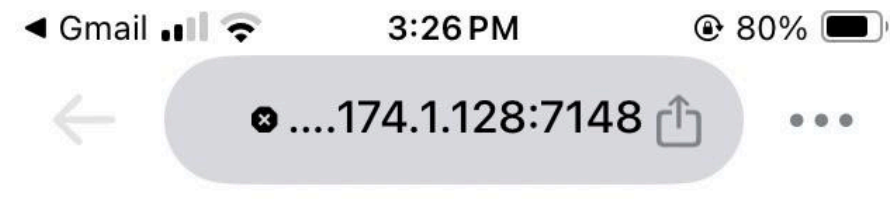
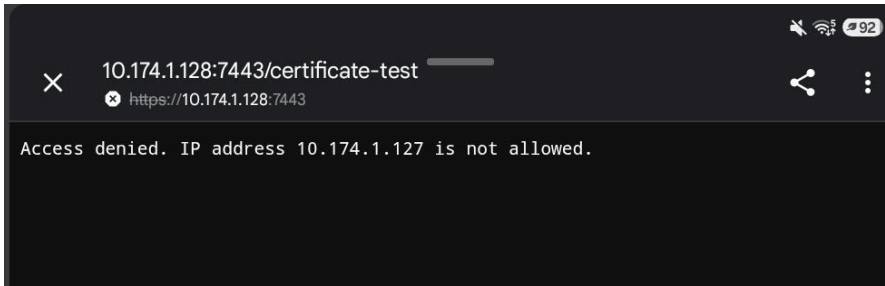


All Tabs

[1 mark] IP Filtering. Capture screen shot indicating your MVC app denying access to unknown IP addresses [as an alternative to client side certificates].

Tested on both cell phones and tablets which were allowed to access while adding the IP addresses into the allowed list in **appsettings.json**. After removing the ip address from the list, both devices unable to access the application were also unable to see the certification information.

```
    },  
    "IPFiltering": {  
      "Enabled": true,  
      "AllowedIPs": [  
        "127.0.0.1",  
        "::1"  
      ]  
    },  
  },  
}
```



Access denied. IP address 10.174.1.122
is not allowed.

[1 mark] Role based authorization as supported by ASP.Net, or your own custom scheme.

In my project, I created four different roles: **Administrator, Manager, Employee, and Client**. Based on the user's role, I authorized different features and views for each role. The main authorization rules are:

1. Only the Administrator can assign the Manager role to an employee.
2. Administrators and Managers can edit and delete information for both employees and clients.
3. Employees can view information about clients and their own profile.
4. Clients can only view their own profile.

The AspNet schema is defined in **ClientContext.cs** since ASP.NET Core Identity automatically provides the model definitions for tables such as:

AspNetUsers
AspNetRoles
AspNetUserRoles
AspNetUserClaims
AspNetRoleClaims
AspNetUserLogins
AspNetUserTokens

In the table AspNetRoles I have defined the four roles and in AspNetUsers defined each user with Id, then in AspNetUserRoles there is a match pair between UserID and RoleId. Each user should either have EmployeeId or ClientId in the database to distinguish the role.

To implement role-based authorization, I define authorization requirements for the relevant controller actions. For example, only Administrators and Managers are authorized to execute the employee deletion action by using the **[Authorize(Roles = "Administrator,Manager")]**.

Similar authorization rules are also applied to views. Some pages and navigation items are only displayed to users with the appropriate roles. However, hiding a page or navigation item is not sufficient for security. Authorization must first be enforced on the server-side functionality, because a user could otherwise directly access a URL without the corresponding navigation item being displayed.

SQL:

```
SELECT * FROMAspNetUsers;
SELECT * FROMAspNetRoles;
SELECT * FROMAspNetUserRoles;
```

100 %

Results Messages

	Id	ClientId	EmployeeId	UserName	NormalizedUserName
1	1eb14abb-83cc-4d27-8b0e-ca7fcec78c0c	NULL	064ED613-4558-44D3-88A9-797D1E8B7909	employee2@example.com	EMPLOYEE2@EXAMPLE.COM
2	2eaca160-dbc6-4cb7-8ed8-9ca009a250e	NULL	93831E90-816C-48B5-8E55-7E56236A03A9	managertest1@example.com	MANAGERTEST1@EXAMPLE.COM
3	43a59cd0-9fef-418b-bbb5-b829321cbe7c	NULL	C83F2CA9-3A81-4567-A3D8-FF570274F635	employee1@example.com	EMPLOYEE1@EXAMPLE.COM
4	596df765-ccaa-4f69-9e3a-5a198fead620	DA1782E7-F62E-43C2-992F-5593664454A5	NULL	testuser2@colleen1005yahoom.onmicrosoft.com	TESTUSER2@COLLEEN1005YAHOO.COM.ONMICROSOFT.COM
5	9ca72e29-c919-40b9-bdbb-60779bb63d89	NULL	NULL	admin@test.local	ADMIN@TEST.LOCAL
6	b4c800ca-38e0-4542-83f1-46da0500294	C641C087-91C0-4D44-8795-D0F5F3C29543	NULL	client1@example.com	CLIENT1@EXAMPLE.COM
7	bac9a3a0-b0c7-4fde-be00-3eb9e09f3925	A4EC621D-0F0D-46A4-A797-543350114461	NULL	testuser@colleen1005yahoom.onmicrosoft.com	TESTUSER@COLLEEN1005YAHOO.COM.ONMICROSOFT.COM

	Id	Name	NormalizedName	ConcurrencyStamp
1	37568a5e-3b04-4121-8c60-330515265e54	Manager	MANAGER	NULL
2	5d17a91d-453e-445e-9d5f-0ac5be121b05	Client	CLIENT	NULL
3	bc3ef1e4-87cc-4ffd-bf2b-8f9345affec3	Administrator	ADMINISTRATOR	NULL
4	e81439df-37a6-4580-b520-791ca651e62c	Employee	EMPLOYEE	NULL

	UserId	RoleId
1	2eaca160-dbc6-4cb7-8ed8-9ca009a250e	37568a5e-3b04-4121-8c60-330515265e54
2	43a59cd0-9fef-418b-bbb5-b829321cbe7c	37568a5e-3b04-4121-8c60-330515265e54
3	596df765-ccaa-4f69-9e3a-5a198fead620	5d17a91d-453e-445e-9d5f-0ac5be121b05
4	b4c800ca-38e0-4542-83f1-46da0500294	5d17a91d-453e-445e-9d5f-0ac5be121b05
5	bac9a3a0-b0c7-4fde-be00-3eb9e09f3925	5d17a91d-453e-445e-9d5f-0ac5be121b05
6	9ca72e29-c919-40b9-bdbb-60779bb63d89	bc3ef1e4-87cc-4ffd-bf2b-8f9345affec3
7	1eb14abb-83cc-4d27-8b0e-ca7fcec78c0c	e81439df-37a6-4580-b520-791ca651e62c

ClientContext.cs

```
namespace MVP_1B2
{
    27 references
    public class ClientContext : IdentityDbContext<ApplicationUser>
    {
        7 references
        public DbSet<Person> Persons { get; set; }
        17 references
        public DbSet<Client> Clients { get; set; }
        12 references
        public DbSet<Employee> Employees { get; set; }
        21 references
        public DbSet<Service> Services { get; set; }
        4 references
        public DbSet<ClientService> ClientServices { get; set; }

        0 references
        public ClientContext(DbContextOptions<ClientContext> options) : base(options) { }

        0 references
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<Person>().ToTable("People");
            modelBuilder.Entity<Client>().ToTable("Clients");
            modelBuilder.Entity<Employee>().ToTable("Employees");
            modelBuilder.Entity<Service>().ToTable("Services");
            modelBuilder.Entity<ClientService>().ToTable("ClientServices");
        }
    }
}
```

The authorization defined in the controller.

```
// GET: Employees/Delete/5
[Authorize(Roles = "Administrator,Manager")]
0 references
public async Task<IActionResult> Delete(Guid? id)
{
    if (id == null)
    {
        return NotFound();
    }
}
```

Views define the authorized roles.

```
MVP_1B2
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59

@if (User.IsInRole("Administrator") || User.IsInRole("Manager"))
{<li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Employees" asp-action="Index">Employee</a>
</li>
}

@if (User.IsInRole("Client"))
{
    <li class="nav-item">
        <a class="nav-link text-dark" asp-controller="Clients" asp-action="MyProfile"> My Profile </a>
    </li>
}

@if (User.IsInRole("Administrator") || User.IsInRole("Manager") || User.IsInRole("Employee"))
{
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-controller="Clients" asp-action="Index">Client</a>
    </li>
}
```

Trying to access unauthorized pages will be forbidden.



[1 mark] Identify any SQL Server assets/files that you would like to protect against unauthorized access and show what file system permissions you will use to prevent that access.

The SQL Server assets that should be protected include the following files:

1. The database files (.mdf) - Primary database file, containing application data
2. Transaction log files (.ldf) - SQL Server transaction log
3. Database backup files (.bak) - Database backup
4. SQL Server configuration or credential files

In my application, the database contains sensitive client, employee, account, and service information, so users should not be able to access these files directly through the Windows file system. Instead, we should only allow users access the database only through the ASP.NET Core application. This prevents users from bypassing application-level authentication and authorization by directly accessing or modifying the underlying SQL Server files.

Besides, the database files should be stored in a protected directory with least-privilege file system permissions. The SQL Server service account requires the permissions necessary to read and write the database files, while authorized database administrators may have administrative access. Normal Windows users should have no access to the database and log files. Database backup files should also be restricted to authorized administrators and the required SQL Server backup process.